SIMATS ENGINEERING ASSESSMENT TOOL

Nivetha S(192511269)

CO2 – AT3: Formal Languages and Automata Theory

Question 1 — DFA Minimization for a PIN-Validation Pattern

Problem: A DFA is used to validate PIN strings such as bbbc, bca, c, ca, caaaaa. Minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems.

Step 1: Identifying the Language

Examining the sample accepted strings — bbbc, bca, c, ca, caaaaa — each string consists of zero or more b's, followed by exactly one c, followed by zero or more a's. So the language recognized is:

$L = \{ b^m c\, a^n \mid m \geq 0, n \geq 0 \}$ $\quad$ over alphabet $\Sigma = \{a, b, c\}$

Step 2: An Unminimized (Given) DFA

A straightforward, non-optimized design for this checker often ends up using extra states that behave identically to one another. Consider the following 5-state DFA that accepts L (q0 = start; q3 = accepting):

State	on a	on b	on c
→ q0	q4	q0	q3
q1	q4	q1	q3
q2	q4	q2	q3
*q3	q3	q4	q4
q4 (trap)	q4	q4	q4

Here q0, q1 and q2 all loop on b and move to q3 on c and to the trap q4 on a — i.e. they are behaviorally identical (redundant) copies of the same "reading b's" state. Such duplication commonly arises when a DFA is built incrementally or generated automatically without simplification.

Step 3: Minimization (Table-Filling / Partition Refinement)

Apply the standard equivalence-partitioning algorithm:

- Initial partition by finality: P0 = { q0,q1,q2,q4 } (non-accepting) and { q3 } (accepting).
- Refine P0 using transitions: q0, q1, q2 each go to {q3} on 'c' and to {q4} on 'a', and stay within the same class on 'b' — so q0, q1, q2 are indistinguishable and stay merged. q4 goes to itself

on every symbol, and is distinguishable from {q0,q1,q2} because q4 can never reach q3, so q4 forms its own class.

- No further splitting is possible ⇒ the algorithm converges with 3 final classes: A = {q0,q1,q2}, B = {q3}, C = {q4}.

Step 4: Minimized DFA

Renaming A → S0 (start), B → S1 (accepting), C → Strap:

State	on a	on b	on c
→ S0	Strap	S0	S1
*S1	S1	Strap	Strap
Strap	Strap	Strap	Strap

The minimized DFA has only 3 states (down from 5) and still accepts exactly L = { b^m c a^n }. It is minimal because S0, S1 and Strap are pairwise distinguishable — S1 is the only accepting state, and Strap is the only state from which the accepting state can never be reached.

Step 5: How State Reduction Improves Performance in Embedded Systems

- Smaller memory footprint: fewer states means a smaller transition table, which matters directly on microcontrollers with limited flash/ROM and RAM.
- Faster execution: each input symbol requires only one table lookup; fewer states mean a smaller table that is more likely to fit in cache, reducing lookup latency.
- Lower power consumption: fewer state comparisons and smaller memory accesses translate into fewer CPU cycles, which is critical for battery-powered embedded devices (e.g., keypad/PIN validators, access-control units).
- Easier verification and testing: a minimal DFA has no redundant states, so exhaustive testing of all states/transitions requires fewer test cases, reducing validation effort and firmware certification time.
- Simpler hardware realization: when the DFA is implemented as digital logic (e.g., in an FSM controller on an FPGA/ASIC), fewer states mean fewer flip-flops for state encoding and simpler next-state logic, reducing silicon area and gate delay.

Question 2 — Converting a Finite Automaton to a Regular Expression

Problem: Demonstrate how the given finite automaton can be converted into a regular expression.

Step 1: Reading the Transition Diagram

From the given diagram (start state v1, accepting/absorbing state v4 with a self-loop on 0 and 1), the transitions are:

From	On 0	On 1
→ v1	v3	v2

v2	v4	v1
v3	-	v1, v4
*v4	v4	v4

v1 is the start state and v4 (the state with the 0,1 self-loop) is taken as the final state.

Step 2: Set Up Arden's Equations

For each state, write an equation expressing it in terms of the states that transition into it (R denotes the regular expression of all strings that reach that state from the start):

$R1 = \varepsilon + R2 \cdot 0 + R3 \cdot 1$ (v1 is the start state, so add ε)
$R2 = R1 \cdot 1$
$R3 = R1 \cdot 0$
$R4 = R2 \cdot 0 + R3 \cdot 1 + R4 \cdot (0+1)$

Step 3: Solve Using Arden's Theorem

Substitute R2 and R3 into the equation for R1:

$R1 = \varepsilon + (R1 \cdot 1) \cdot 0 + (R1 \cdot 0) \cdot 1 = \varepsilon + R1(10) + R1(01) = \varepsilon + R1(10+01)$
By Arden's theorem ($X = Q + XR \Rightarrow X = QR^*$), with $Q = \varepsilon$ and $R = (10+01)$:

$R1 = (10+01)^*$
$R2 = R1 \cdot 1 = (10+01)^* 1$
$R3 = R1 \cdot 0 = (10+01)^* 0$

Substitute R2 and R3 into the equation for R4:

$R4 = (10+01)^*1 \cdot 0 + (10+01)^*0 \cdot 1 + R4(0+1)$
$= (10+01)^*(10) + (10+01)^*(01) + R4(0+1)$
$= (10+01)^*(10+01) + R4(0+1)$
Applying Arden's theorem again ($Q = (10+01)^*(10+01)$, $R = (0+1)$):

$R4 = (10+01)^*(10+01)(0+1)^*$

Step 4: Final Regular Expression

Since $(10+01)^*(10+01) = (10+01)^+$ (one or more repetitions), the regular expression accepted by the automaton (start v1 to final v4) is:

$R = (01+10)+(0+1)^*$

In words: any non-empty sequence of alternating blocks "01" or "10", followed by an arbitrary suffix of 0's and 1's — reflecting that the automaton must first move away from v1 through v2/v3 at least once before it can settle permanently into the absorbing accepting state v4.

Question 3 — Grammar Analysis

Grammar:

$S \to aSc \mid T \mid U \mid \varepsilon$
$T \to bTc \mid \varepsilon$
$U \to aUb \mid \varepsilon$

(A) Language Defined by the Grammar

T generates matched b's and c's: $L(T) = \{ b^n c^n \mid n \geq 0 \}$.

U generates matched a's and b's: $L(U) = \{ a^n b^n \mid n \geq 0 \}$.

S → aSc wraps i matching a's/c's around a core derived from T, U, or ε. Combining all three choices:

- Via ε: $a^i c^i$ (p a's, r c's, p = r, q = 0 b's)
- Via T: $a^i b^n c^{(n+i)}$ (p = i, r = n+i, so r ≥ p and q = r − p)
- Via U: $a^{(n+i)} b^n c^i$ (p = n+i, r = i, so p ≥ r and q = p − r)

All three cases collapse into a single clean description — in every case the number of b's equals the absolute difference between the number of a's and the number of c's:

$$L(S) = \{ a^p b^q c^r \mid p, q, r \geq 0 \text{ and } q = |p - r| \}$$

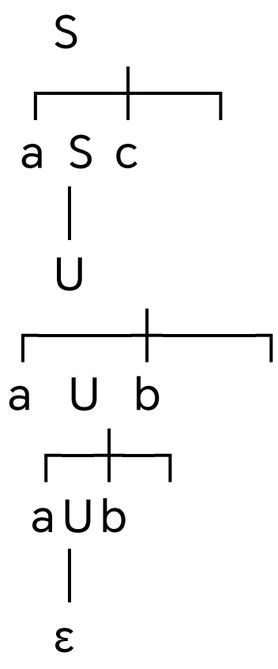
(B) Derivation Tree for aaabbc

For aaabbc: p = 3 a's, q = 2 b's, r = 1 c. Since p > r, the U-branch is used, with i = r = 1 outer application of S → aSc, and n = p − r = 2 applications of U → aUb before U → ε.

Leftmost derivation:

$$S \Rightarrow aSc \Rightarrow aUc \Rightarrow a(aUb)c \Rightarrow a(a(aUb)b)c \Rightarrow a(a(a \cdot \varepsilon \cdot b)b)c = aaabbc$$

Derivation tree (read leaves left to right to recover the string aaabbc):



Leaves left to right: a, a, a, ε, b, b, c = a a a b b c = aaabbc. ✓

(C) Is the Grammar Ambiguous?

Yes — the grammar is ambiguous. Although the string aaabbc itself happens to have only one derivation (because p ≠ r forces the U-branch), other strings in the language admit more than one distinct derivation tree, which is enough to make the grammar ambiguous.

Counter-example: consider the string "ac" (p = 1, q = 0, r = 1, and p = r). Because S can dispatch directly to ε, or to T (which also reduces to ε), or to U (which also reduces to ε), there are three distinct derivations of the same string:

$$\text{Derivation 1: } S \Rightarrow aSc \Rightarrow a\cdot\varepsilon\cdot c = ac \qquad (S \to \varepsilon \text{ directly})$$
$$\text{Derivation 2: } S \Rightarrow aSc \Rightarrow aTc \Rightarrow a\cdot\varepsilon\cdot c = ac \qquad (S \to T, \text{then } T \to \varepsilon)$$
$$\text{Derivation 3: } S \Rightarrow aSc \Rightarrow aUc \Rightarrow a\cdot\varepsilon\cdot c = ac \qquad (S \to U, \text{then } U \to \varepsilon)$$

These three derivations produce three structurally different parse trees (the middle child of S is, respectively, an ε-node, a T-node, and a U-node) for the identical string "ac". Since a single word has more than one distinct derivation tree, by definition the grammar is ambiguous.

Question 4 — Closure and Regularity

(i) If L1 and L2 are regular, then L1 ∪ L2 is regular

Proof (via regular expressions): Since L1 and L2 are regular, there exist regular expressions r1 and r2 such that L1 = L(r1) and L2 = L(r2). By the definition of regular expressions, if r1 and r2 are regular expressions, then (r1 + r2) is also a regular expression, and it denotes exactly L(r1) ∪ L(r2) = L1 ∪ L2. Hence L1 ∪ L2 is a regular language.

Proof (via automata, equivalently): Let M1 = (Q1, Σ, δ1, q1, F1) and M2 = (Q2, Σ, δ2, q2, F2) be DFAs/NFAs with L(M1) = L1 and L(M2) = L2. Construct an NFA M with a new start state q0 and ε-transitions from q0 to q1 and to q2, taking the (renamed, disjoint) states of M1 and M2 together with F1 ∪ F2 as the accepting states. M accepts w iff w is accepted by M1 or by M2, so L(M) = L1 ∪ L2. Since every NFA has an equivalent DFA, L1 ∪ L2 is regular.

(ii) Is L = { a^p | p is prime } regular?

No, this language is not regular. This is proved using the Pumping Lemma for regular languages.

Suppose, for contradiction, that L is regular. Let n be the pumping length guaranteed by the pumping lemma. Since there are infinitely many primes, choose a prime p ≥ n. Let s = a^p , so |s| = p ≥ n.

By the pumping lemma, s can be written as s = xyz with |xy| ≤ n, |y| ≥ 1, and $xy^iz \in L$ for every i ≥ 0. Let |y| = k, where 1 ≤ k ≤ n. Then:

$$|xy^iz| = p + (i-1)k \quad \text{for every } i \geq 0$$

Choose i = p + 1. Then the resulting length is:

$$|xy^{(p+1)}z| = p + p\cdot k = p(1+k)$$

Since p ≥ 2 and k ≥ 1, both factors p and (1+k) are integers ≥ 2, so p(1+k) is a composite number — it cannot be prime. Therefore $xy^{(p+1)}z = a^{(p(1+k))} \notin L$, contradicting the pumping lemma's guarantee that $xy^iz \in L$ for all i ≥ 0.

This contradiction shows that L = { a^p | p is prime } is not regular.